

「読めない」を分解する

Arduino Nano 3台によるスーパーファミコン
カートリッジ読み出し装置の製作と、SA-1攻略の報告

2026年8月 実測記録にもとづく報告

要旨

追加部品を一切購入しないという制約のもとで、Arduino Nano 3台によるスーパーファミコン（SFC）カートリッジ読み出し装置を製作した。アドレス下位・データバス制御・バンクアドレスの3役を分担させ、レベル変換ICもシフトレジスタも用いずに34本以上の入出力要求を満たしている。17タイトルの読み出しに成功し、全ROMについてヘッダ申告値と全バイト総和の一致を確認した（うち8タイトルはNo-IntroのCRC32とも照合済み）。

本報告の主題は成功手順ではなく、「読むたび内容が変わる」という単一の症状が、電源不足・接触不良・タイミング余裕不足・バス調停という4つの異なる原因から生じるという点にある。これらを区別する手続きとして、「読み出しタイミング段階に対する相違バイト数の単調性」と「0xFF率の安定性」の2指標を提案し、実測で検証した。

あわせて、SA-1搭載カートリッジ2タイトルの読み出しに成功したことを報告する。**必要だったのは、CIC認証を通したうえで、カートリッジにクロックを一切与えないことだった。**SA-1はクロックを与えれば起動してバスを掌握するが、与えなければ眠ったままでROMは素通しで読める。文献が必須とするマスタークロックは不要だったが、CIC認証は必要だった。なお本報告は当初この点を誤って「CIC認証も不要」と結論しており、その誤りの原因（実験順序による交絡）も7章に記録した。セーブデータは逆にSA-1が動作していなければ読めず、未達である。

表1 装置と成果の概要

項目	内容
構成	Arduino Nano ×3（すべて5Vネイティブ）＋ジャンク実機から摘出した62ピンコネクタ
追加購入部品	なし（設計上の制約）
読み出し速度	1バイト47us（初期実装331usから7.0倍）／2MBあたり約3分
検証済みタイトル	17（総和一致17／No-Intro CRC32照合8）
対応した特殊チップ	Super FX（GSU / GSU-2）、DSP-1、 SA-1（CIC認証あり）
未達	SA-1カートの セーブデータ （ROMは取得済み）
ソフトウェア	GUI（Tkinter）／CLI／オフラインDB検索（No-Intro準拠7,621件）

本報告は、うまくいった手順よりも**外れた仮説の記録に紙面を割いている**。誤診はいずれも「もっともらしい説明が先に立ち、それを検証せずに次の実験へ進んだ」という同じ形をしており、そこに再利用可能な教訓があると判断したためである。

1. 序論

1.1 背景

SFCのカートリッジは、62ピンのカードエッジコネクタを介してROMをアドレス空間へ直接露出している。したがって外部からアドレスを与え、制御線进行操作し、データバスを読み取れば、原理的にはROMの内容を取り出せる。この操作を実装した装置は複数存在し、事実上の標準実装は sannu の Open Source Cartridge Reader（以下 OSCR）である。

本プロジェクトの出発点は「ESP32-S3の開発ボードとジャンクのスーパーファミコンが手元にある。これだけで吸い出せないか」であった。そこに「**追加で一つも部品を購入しない**」という制約を課した。この制約は単なる縛りではなく、以降の設計をほぼすべて決定している。

1.2 入出力要求と構成の選定

SFCカードの読み出しに必要な信号は、アドレス24本・データ8本・制御2本、電源系を除いて**最低34本**である。この一点で候補の大半が脱落した。

表2 構成候補と却下理由

構成	I/O	判断
ESP32-S3 単体	十分	却下 。5Vトレラントでない。カードの5V出力を直結すると破損する。レベル変換ICが必須だが手持ちがない
Arduino Uno 単体	18本	却下 。34本に遠く及ばない
Arduino Mega 2560	54本	却下 。直結でき最も素直（OSCRがこの構成）。ただし買い足しが要る
Raspberry Pi Zero 2 + Nano	十分	却下 。3.3V系のためデータバス8本+ACKに分圧抵抗が18本ほど必要になり、「手持ちだけで完結」という利点が消える
Arduino Nano ×3	60本	採用 。全ボードが5Vネイティブで電圧問題が原理的に発生しない。追加部品ゼロ

採用理由は消去法である。しかしこの選択は、後述するようにCIC認証への取り組みにおいて予期しない構造的優位を生んだ（7章）。

1.3 本報告の構成

2章で装置の構成を述べる。3章で読み出しプロトコルと検証手法を整理し、4章で誤診の記録を扱う。ここが本報告の中核である。5章で性能、6章で検証結果を示す。7章はSA-1に関する未達の報告であり、8章で全体の考察を述べる。

2. 装置の構成

2.1 役割分担

3台のNanoに、アドレス下位・バス制御とデータ読み取り・バンクアドレスを分担させた。Nano-2がマスタとして動作し、PCとのUSBシリアル通信を独占する。Nano-1とNano-3はPCと通信せず、Nano-2からのSTROBEパルスでカウンタを進めるだけの自律動作をとる。

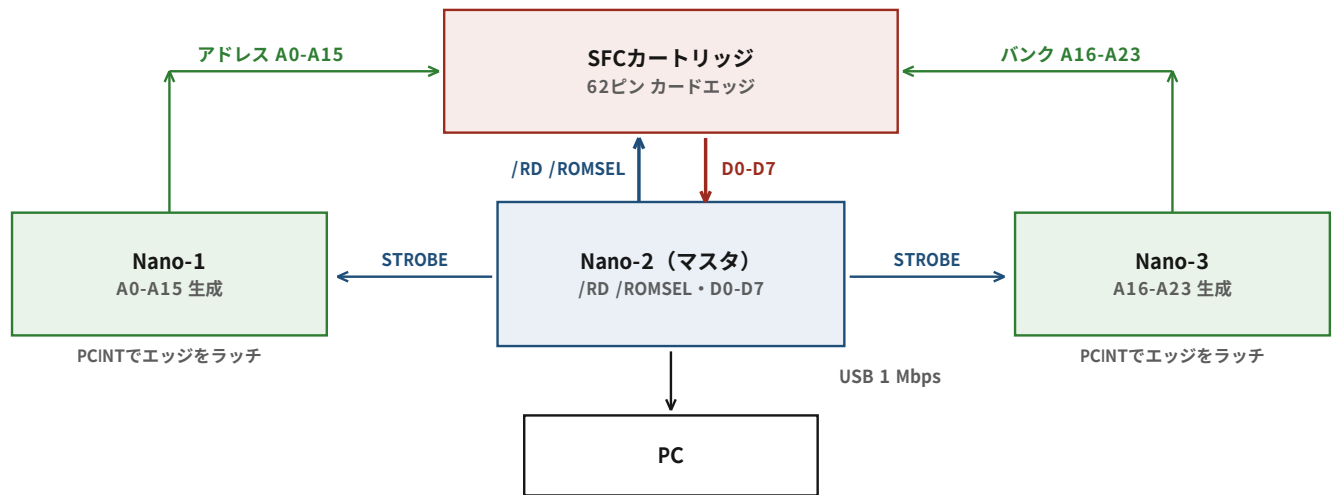


図1 装置のブロック図。Nano-1／Nano-3はPCと通信せず、STROBEの立ち上がりで内部カウンタを+1する。

表3 各ボードの担当

ボード	担当	実装上の要点
Nano-1	カートアドレス A0-A15	STROBEの立ち上がりで16bitカウンタを+1、RESETで0クリア。ポート直叩き・PCINT割り込み・millis()停止まで実装済み
Nano-2	/RD・/ROMSEL制御、D0-D7読み取り、USB送信、OLED表示	唯一PCと通信する。制御ピンはPORTDにまとめてマクロ化
Nano-3	カートアドレス A16-A23（バンク）	Nano-1と同一方式。SA-1実験ではCIC役を兼務させた（8章で述べる問題を起こした）

/WR はカート側で +5V に直結している。読み出し専用として設計したため、ファームウェアに何を書いてもSRAMへの書き込みは物理的に発生しない。この配線は、後にバッテリーバックアップSRAMの吸い出し機能を追加したとき、そのまま救出対象を守る安全装置として働いた。

2.2 コネクタという物理的な壁

62ピンのカードエッジは基板の両面に31本ずつ端子が並ぶ形式であり、表裏で124本ではない。この装置では、ジャンクのスーパーファミコン本体からコネクタを取り外して流用した。これが「買わずに済ませた」最大の部品である。

当初「ピッチが2.0mmだから31ピンで大きくずれる」と説明したが、これは誤りだった。「それなら20ピンで1cmずれる、刺さるはずがない」という指摘を受けて資料を当たり直したところ、実際のピッチは**2.50mm**で、2.54mmとの差は0.04mm、31ピンでも累積1.2mm程度に留まる。整合しない主因はピッチではなく**列間隔とキー溝の位置が非標準**であることだった。最終的にキー溝を超音波カッターで切断し、金属シールドで位置決めして押し込んでいる。

記録: 数字が合わないと指摘されたときは、自分の記憶ではなく一次資料を当たる。この誤りは、指摘がなければそのまま設計判断に影響していた。

3. 読み出しプロトコルと検証手法

3.1 メモリマップとミラー

SFCのROMマッピングにはLoROMとHiROMがあり、ヘッダの map mode バイトが自己申告している（0x20=LoROM、0x21/0x31=HiROM、0x30=LoROM+FastROM、0x23=SA-1）。**LoROMではA15がROM選択に関与しないため、1バンク64KBの上位32KBと下位32KBに同じ内容が現れる。**これは故障ではなく正常な電氣的挙動である。

この性質は「後半のバンクが前半のミラーになる」と誤解されやすいが、ミラーは**バンクの中で**起きるのであってバンク単位ではない。2MBのLoROMを読むには32KB×64バンクが必要であり、32バンクでは足りない。



図2 LoROMのミラーは1バンクの内部で起きる。バンク単位のミラーではない。

3.2 チェックサム検証の限界

SFCの内部ヘッダにはチェックサムとその補数が格納されており、2つを足して0xFFFFになるかを検査できる。**しかしこれはヘッダの4バイトが正しく読めたことしか保証しない。**ROM本体が壊れていても通る。

初期にSuper Puyo Puyoの1MBダンプでこの検査が通ったため「成功」と判断したが、誤判定だった。エミュレータ（Snes9xコア）に読ませたところ **Invalid Checksum** が出力された。正しい検証は**ROM全体のバイト総和**である。

```
checksum = rom[off+30] | (rom[off+31] << 8) # ヘッダが主張する値
computed = sum(rom) & 0xFFFF # 実際に計算した値
```

以降、この総和とNo-IntroのCRC32の両方を検証に用いている。

3.3 2の累乗でないROMの扱い

ヘッダのROMサイズバイトはlog2表現であり、**2の累乗でしか表現できない**。したがって12Mbit（1.5MB）のカートリッジは2MBと名乗る。このとき後半4Mbitはアドレス空間に2回現れるため、総和も同じ数え方をしなければならない。

ドラゴンクエストI・IIを読んだとき、64バンクすべてが未決着0バイト（＝5サンプルが全バイトで一致）だったにもかかわらず総和が合わなかった（計算0x47b5／期待0x2d8a）。読みが完全に安定している以上、犯人は読み出しではない。

```
sum(前半1MB) + 2 × sum(後半512KB) = 0x2d8a → 期待値と一致
CRC32 = 98bb6853 → No-Intro "Dragon Quest I & II (Japan)" / 1,572,864 bytes と一致
```

本報告の作成にあたり、かまいたちの夜（3MB）についても同じ検算を行い、sum(前半2MB) + 2 × sum(後半1MB) = 0x815c で期待値と一致することを確認した。

「チェックサムが合わない＝読めていない」ではない。読み出しの品質と、サイズの解釈は別々に疑う必要がある。

4. 誤診の記録 — 同じ症状、異なる原因

本章が報告の中核である。「読むたび内容が変わる」という症状は、少なくとも4つの独立した原因から生じる。そして同じ原因を当てはめると必ず間違える。それを実際に3回繰り返した。

4.1 ストロブの取りこぼし — 最も長く誤診した問題

高速化のため `digitalWrite/digitalRead` (1回約4us) をポートレジスタ直叩きへ置き換え、ボーレートを1Mbpsに上げたところ、速度は12倍になったがデータが壊れた。

表4 最初に立てた仮説と、その顛末

仮説	検証結果
ポート直叩きで同時スイッチングノイズが増えた	アドレスは毎回+1なので遷移するビットは平均2本程度。説明として苦しい
長時間の連続動作による電圧降下	外部電源に替えても変化なし
デカップリング不足	最短距離で実装済み・33uF搭載済み・GNDも両方引いている。打つ手は尽きていた

決定打は、同じバンクを10回読んで**毎回きっかり30,966バイトの相違**が出たことだった。ノイズであれば数値はばらつく。完全に決定的である。そこで「どう壊れているか」を総当たりで検算した。

表5 出力と正解の対応づけを総当たりで検算した結果

仮説	一致率
そのまま 出力[i] == 正解[i]	5.5%
2倍速 出力[i] == 正解[2i]	66.7%
半分（重複読み） 出力[i] == 正解[i÷2]	100.0% (32,768/32,768)

同じアドレスを2回ずつ読んでいた。Nano-1がストロブをきっかり1回おきに取りこぼしていたのである。ポーリングには「ループを一周する間に来たパルスを取りこぼす」という原理的な穴があり、Nano-1の処理時間がNano-2の1サイクルより長くなれば**必ず半分落ちる**。パルス幅を60us、100usと広げていたのは、穴を塞がずに誤魔化していただけだった。

解決はNano-1／Nano-3の**ピン変化割り込み（PCINT）**化である。エッジをハードウェアがラッチするため、ISRが動く前にパルスが終わっていても取りこぼさない。穴そのものが消える。

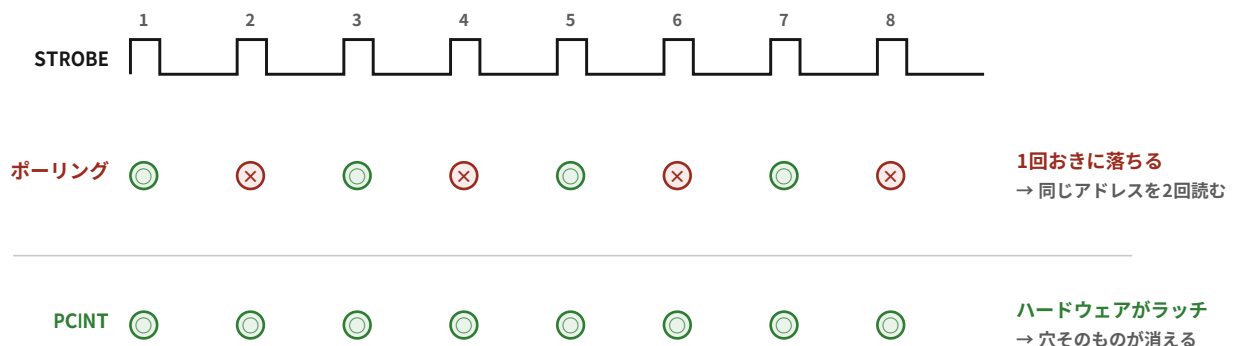


図3 ポーリングは処理時間が1サイクルを超えた瞬間から半分を落とす。PCINTはエッジをハードウェアで保持するため、この失敗モードが存在しない。

教訓: 数値が毎回びたりと同じなら、それはノイズではなく構造的な誤りである。「壊れている」で止めずに「どう壊れているか」を測ったことが突破口になった。

4.2 スターフォックス — 犯人は電源だった

Super FX (GSU) 搭載カートは長らく読めなかった。同じバンクを読むたび数百バイトが化け、2回一致が永久に成立しない。多数決ダンプを実装して10サンプルを積んだが、1MB中38,937バイトが決着せずCRCも合わなかった。

誤診に至った理由は、症状が「動作中のバスマスタと衝突している」ようにしか見えなかったことである。特に**遅くするほど悪化する**という挙動が決定的な誤解を招いた。「読み出しに時間をかけるほどGSUが割り込む機会が増える」と解釈したが、実際は**遅い設定ほど1バンクの読み出しが長引き、弱い電源を長時間虐めていただけだった**。

表6 安定化電源を5Vバスへ直結する前後の比較 (同一バンクの2回読みの相違バイト数)

段階	1バンクの所要	改善前	改善後
高速 (5us)	15.0 s	89 / 75	0
やや低速 (20us)	19.2 s	338 / 611	0
低速 (50us)	27.8 s	5,259 / 2,758	0
安全 (100us)	42.3 s	1,969 / 1,340	0
最安全 (300us)	99.1 s	2,524 / 9,828	29

電源改善後、同じ設定で連続5回読んで1バイトも違わなくなった。その状態でLoROM

32バンクを読んだところ、総和もCRC32も一発で一致した (Star Fox: 41a60b3f)。**原因はGSUではなく電源だった**。

この結果は自動化機構の前提を壊した。本装置のタイミング自動エスカレーション (5→20→50→100→300us) は「遅くすれば安全になる」を前提にしているが、その前提は電源が健全なときにしか成り立たない。電源が弱いと**自動エスカレーションが自ら傷口を広げる**。

4.3 ヨッシーアイランド — 今度は接触だった

次のSuper FXカート (GSU-2, 2MB) で、まったく同じ症状が出た。電源が犯人だった直後なので当然そちらを疑ったが、外れだった。

表7 ヨッシーアイランドで疑ったものと検証結果

疑ったもの	検証結果
電源不足	× 安定化電源が4.99Vを維持
発熱	× 触っても熱くない
アドレス線・データ線の不良	× どのビットにも偏りなし (各3.4~6.6%)
カートリッジの接触	○ 挿し直して0xFFが12分の1、連続3回読みで相違0

決め手は**数分の間に品質が12倍変動したこと**である (上位32KBの0xFFが15,306個から1,268個へ)。電氣的な設計問題であればこのような不連続な揺れ方はしない。接触であれば当然である。持ち主の「実機で遊んでいた頃から調子の悪いカセット」という証言が、こちらの測定より早く正解に近かった。

4.4 かまいたちの夜 — タイミング余裕不足

LoROM 3MBのこのカートは、連続2回読みで上位32KBが2,271バイト相違し、0xFF率が16.8%あった。接触不良にしか見えない。しかし段階を掃引したところ、相違が段階に対して単調に減り、かつ**0xFF率がまったく動かなかった**。

表8 かまいたちの夜のタイミング段階掃引

読み出しタイミング	バンク\$00 相違	バンク\$28 相違	0xFF率
高速 (5us)	3,238	1,502	0.1675
やや低速 (20us)	1,028	239	0.1678
低速 (50us)	438	51	0.1678
安全 (100us)	110	7	0.1677
最安全 (300us)	1	0	0.1678

0xFF率が段階によらず0.1675～0.1678に留まることが、その0xFFが本物のデータであることの証拠になる。読み落としであれば段階を変えれば率が動く。接触不良であれば跳ね回る。

4.5 切り分け手順の確立

以上3件から、2つの指標で原因を分離できることが分かった。

指標	意味
相違バイト数の、段階に対する単調性	単調に減る＝タイミング不足／無関係＝接触／遅くすると増える＝電源
0xFF率の安定性	不動＝その0xFFは実データ／跳ね回る＝接触で読み落としている

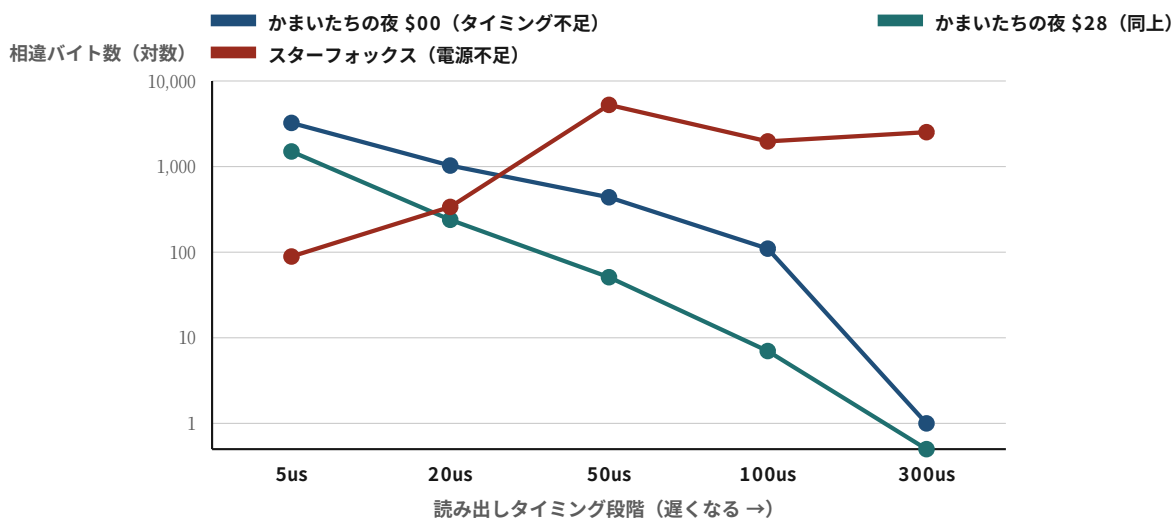
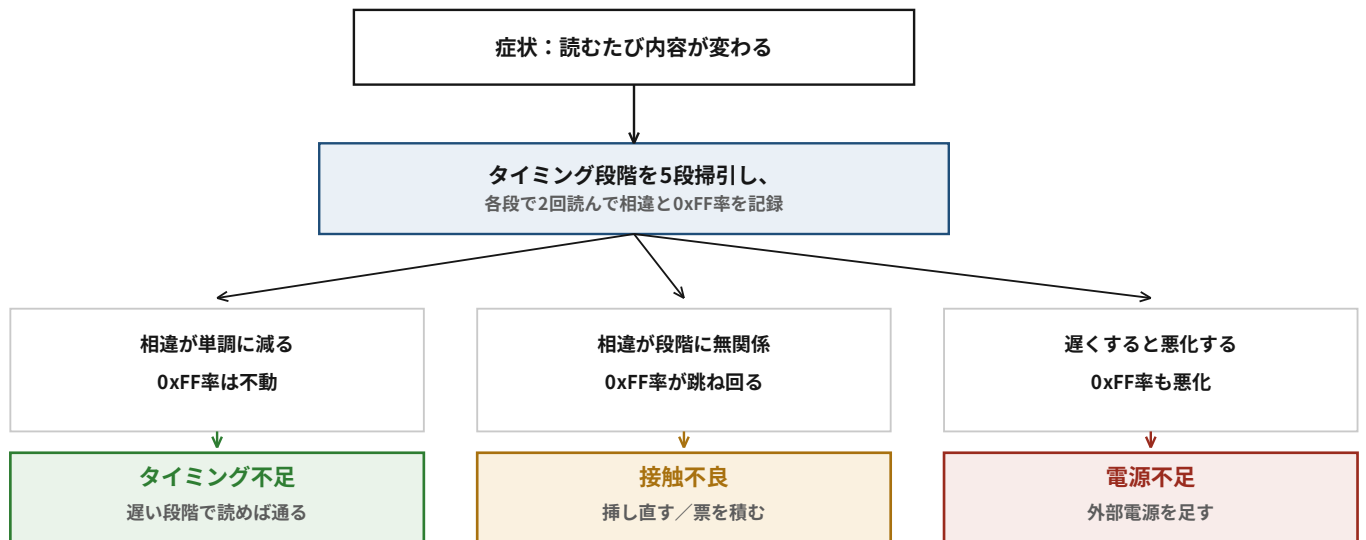


図4 同じ「読むたび化ける」でも、段階に対する応答は逆向きになる。かまいたちの夜は単調減少（タイミング不足）、スターフォックスは遅くするほど悪化（電源不足）。



同じ症状に同じ原因を当てはめると必ず間違える。2つの指標で分ける。

図5 確立した切り分け手順。GUIの「タイミング掃引」ボタンとして実装した。

4.6 検証手法そのものが持つ穴

「2回読んで一致すれば正しい」という検証には落とし穴がある。**カートが外れて全バイト0x00になった場合、その出力は自分自身と必ず一致する。**同様に0xFF一色の読みも安定していて一致する。そのため全バイトが同一値のデータは異常として弾いている。

さらに深い問題として、**キャッシュ機構はカートの同一性を確認していなかった。**かまいたちの夜を読み終えたあと、以前の実行が残っていたキャッシュを照合したところ、32,768バイト中32,191バイトが相違した。ずれや化けではなく、**中身がスターフォックスだった**（ヘッダのタイトルがSTAR FOX、先頭256バイトが別ファイルのオフセット0と完全一致）。現在はキャッシュ再利用時に開始バンクを1本だけ読んで突き合わせ、食い違えばその場で停止する。

5. 性能 — 何が律速か

5.1 高速化の経過

初期実装は331us/バイト（2MBで約12分）だった。内訳はほぼ全部が待ち時間であり、ROMの実力（アクセスタイム120～200ns）から見て500倍以上過剰である。4.1節のPCINT化ののち、待ち時間を段階的に掃引して詰めた。判定基準は**各条件で2回読み、両方とも正解と完全一致した場合のみ合格**とした。

この基準は必要だった。実際PULSE=60usは単発では完全一致するのに、繰り返すと数千バイト化けている。

表9 待ち時間の掃引結果（正解既知のカートで照合）

設定 (RD / ADDR / PULSE)	1バイト	判定
20 / 20 / 5	79 us	合格
10 / 10 / 3	57 us	合格
5 / 5 / 3	47 us	合格 ← 採用
3 / 3 / 2	42 us	合格
2 / 2 / 2	40 us	合格
1 / 1 / 1	38 us	不合格（ヘッダが化ける）

最速値ではなく1段手前を採用したのは、カートの接触状態のばらつきに対する余裕のためである。

5.2 律速の分解

後日、SA-1攻略のためさらなる高速化を検討した際、「Nano-1が16本のdigitalWriteをしているのでそこが本丸」という見込みを立てたが**誤りだった**。実装を読むと、ポート直叩き・PCINT・millis()停止まですべて済んでいた。誤解の元は次のコメントである。

```
delayMicroseconds(addrSettleUs); // Nano-1側の16本分のdigitalWrite完了を待つマージン
```

これは直叩き化する前の古い記述が残っていただけで、実コードは3命令しか使っていない。**コメントを鵜呑みにして実装を見なかった**。

そこで待ち時間を振って実測したところ、明確な回帰が得られた。

1バイトあたり所要時間 = 1.011 × 待ち時間 + 34.1 us

傾きがほぼ1なので待ちはそのまま効くが、**問題は切片の34usである**。待ちを0にしても36us/バイトより速くならない。内訳のうちシリアル送信1Mbps 8N1の10us/バイトは物理的な下限であり削れない。残る約26usはArduinoのシリアル実装（リングバッファ、割り込み）、関数呼び出し、ループ処理である。

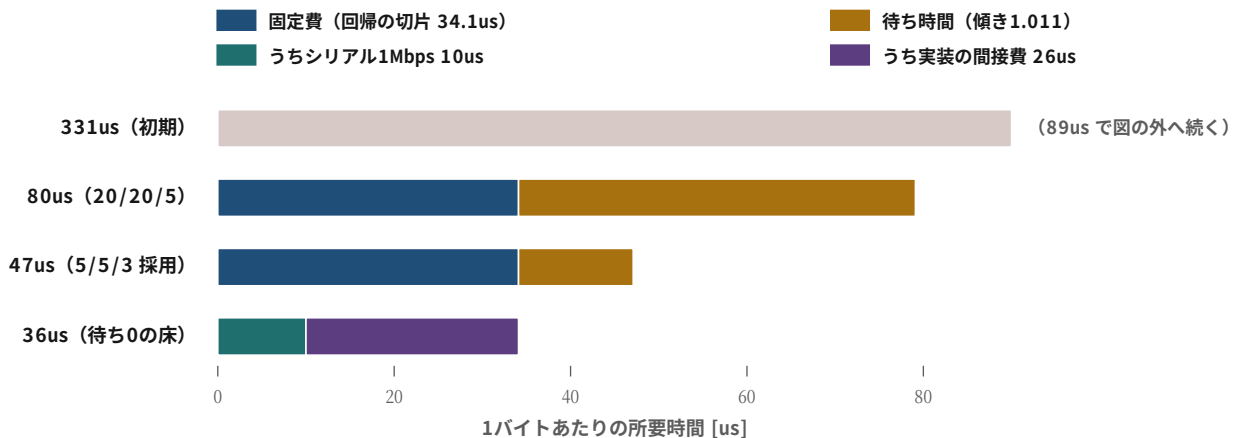


図6 1バイトあたり所要時間の内訳。待ちを0にしても36usの床が残り、その約4割はシリアル送信の物理的下限である。

なお、Nano-2の制御ピンをポート直叩き化する変更も実施したが、計算上24us削れるはずのところ**実測は77.7us→79.6usでほぼ変化がなかった**。見込みが外れた事実として記録しておく。

5.3 OSCRとの構造差

OSCRは1バイトあたり約375ns（アドレスを置いて6NOP待って読むだけ）で読み出す。本装置は約48usであり、**約128倍の差**がある。この差は実装の巧拙ではなく構造に由来する。

表10 読み出し1バイトあたりの構造比較

項目	OSCR (sanni)	本装置
/RD	起動時にLow固定、以後不動	バイトごとにLow→High
/ROMSEL (/CS)	起動時にLow固定、以後不動	バイトごとにLow→High
アドレス生成	PORTF/PORTKへ直接書く	STROBEでNano-1のカウンタを+1
バンク生成	PORTLへ直接書く	STROBEでNano-3のカウンタを+1
データの行き先	SDカードへ直接書き込み	USBシリアルでPCへ (1Mbps)
1バイト所要	約375 ns	約48 us

本装置の構造上、シリアル転送が律速である。OSCRはSDカードへ直接書くためシリアル送信が存在しない。したがって375ns/バイトには原理的に届かない。この128倍の差は、7章で述べるSA-1攻略において決定的な意味を持つ。

6. 結果

2026年8月時点で読み出しに成功したタイトルを示す。総和欄は、ROMサイズを正しく解釈したうえでヘッダ申告値と全バイト総和が一致したことを表す。CRC32欄の印は、記録上No-Introのデータベースと照合したことが確認できるものに限って付けた。

表11 読み出しに成功したタイトル一覧（全ファイルを再検証した）

タイトル	サイズ	map	CRC32	総和	No-Intro
SUPER MARIOWORLD	512 KB	0x20	0ec0ddac	○	
SIMCITY	512 KB	0x20	a39fd8d8	○	
SUPER MARIO KART (DSP-1)	512 KB	0x31	c8002453	○	✓
FINAL FANTASY 4	1 MB	0x20	caa15e97	○	
MARIOPAINT	1 MB	0x20	38c9626c	○	
SUPER PUYOPUYO	1 MB	0x30	9b386875	○	
STAR FOX (Super FX)	1 MB	0x20	41a60b3f	○	✓
DRAGONQUEST 1・2	1.5 MB	0x20	98bb6853	○	✓
DRAGONQUEST5	2 MB	0x20	a19c95d1	○	
SUPERMARIO COLLECTION	2 MB	0x20	232bb5b8	○	✓
Street Fighter 2	2 MB	0x20	5556c5c9	○	
YOSSY'S ISLAND (GSU-2)	2 MB	0x20	343e0dfb	○	✓
夜光虫	2 MB	0x30	0cd0e8d8	○	
KAMAITACHI NO YORU	3 MB	0x20	71c631aa	○	✓
SUPER DONKEY KONG	4 MB	0x31	50f2d1bc	○	
SUPER MARIO RPG (SA-1)	4 MB	0x23	5527071e	○	✓
KIRBY SUPER DELUXE (SA-1)	4 MB	0x23	1f35f230	○	✓

17タイトル、合計約32MB。所要時間は現在の設定で2MBあたり約3分である。初期には1タイトルあたり20回以上のダンプと多数決マージを要していたが（Super Puyo Puyoは27サンプル）、4.1節の解決以降は一発で通るようになった。

6.1 副次的に得られたもの

バッテリーバックアップSRAMの吸い出し。ROMは焼き直せば同じものが手に入るが、セーブデータは世界に1つしかない。しかもカート内のボタン電池は寿命が近い。夜光虫で3スロットぶんのセーブがRetroArchで読み込めることを実機確認した。

実装上の落とし穴は /ROMSEL の極性がマッピングで逆になることだった。SNESの /CART がアサートされる条件は「バンク \$40-\$7D は全アドレス、バンク \$00-\$3F は \$8000-\$FFFF のみ」であるため、LoROM（\$70:0000-\$7FFF）ではアサートし、HiROM（\$20:6000-\$7FFF）ではアサートしない。

DSP-1カートは \$C0 から読む。スーパーマリオカートはHiROM

512KBとして8バンク読めるが総和が合わない（実測0x110e／期待0x8a23）。DSP-1がバンク \$00-\$3F の \$6000-\$7FFF にマップされ、8KB×8バンク=64KB分がROMではなくDSPの応答になるためである。領域ごとに異なるバイト値の種類数を数えると一目で分かる。

領域	異なるバイト値の種類数	解釈
\$4000-\$5FFF	200種	ROM
\$6000-\$7FFF	4種	DSP-1に乗っ取られている
\$8000-\$9FFF	255種	ROM

HiROMではバンク \$C0 以降にもROM全体がマップされ、そちらにDSP-1は居ない。開始バンクを \$C0 に変えるだけで総和もCRC32も一致した。

オフラインのタイトルデータベース。 RetroArch同梱の .rdb（No-Intro準拠、7,621件）を読むMessagePackデコーダを外部ライブラリなしで実装した。3.3節で述べた「ヘッダのサイズ申告が2の累乗に切り上げられる」問題に対し、正確なバイト数を引ける。ネットワークアクセスは不要である。

7. SA-1 — クロックを与えず、認証を通す

SA-1搭載カートリッジ2タイトルの読み出しに成功した。本章は成果の報告であると同時に、**なぜ25日もかかったのか**の分析である。必要だったのは**CIC認証**と、**カートリッジにクロックを一切与えないこと**だった。文献が必須とするマスタークロックは要らなかった。

表12 SA-1搭載カートリッジの読み出し結果

タイトル	サイズ	CRC32	総和	所要	Snes9x
Super Mario RPG (Japan)	4 MB	5527071e	0x4575 一致	95秒	Checksum OK
Hoshi no Kirby Super Deluxe (Japan) (Rev 2)	4 MB	1f35f230	0xf602 一致	95秒	Checksum OK

カービィは電源を入れ直した2回目の読み出しが1回目と**0バイト相違**で、再現性がある。両タイトルともRetroArch（Snes9xコア）が**ROM+RAM+BAT+SA-1**と正しく認識し、エラーなく起動する。

7.1 なぜSA-1が難しいと思われていたのか

SA-1は10.74MHz動作の65816であり、ROMとバスの間に位置するバス調停チップである。この構造から「SA-1を通してもらわなければROMに届かない」と考えられてきた。本報告の初版にも、筆者はそう書いた。

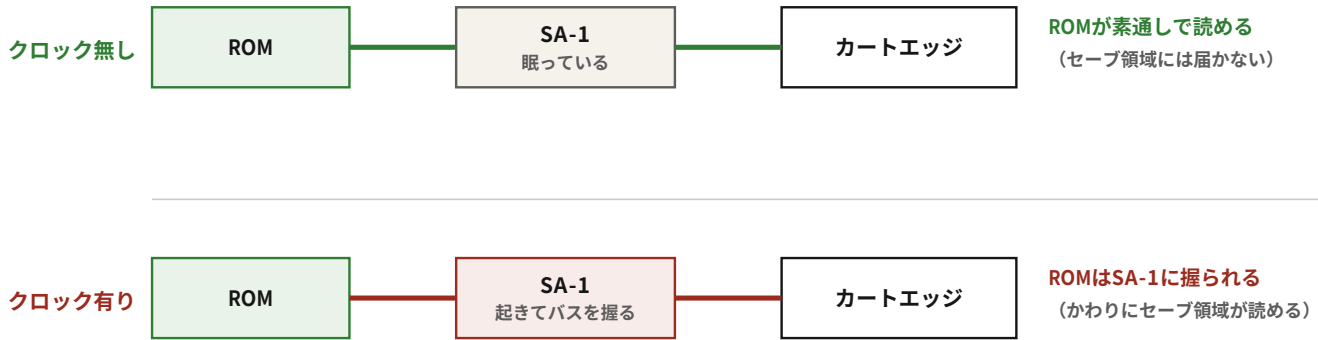
文献も一致してそう述べている。sanniのOpen Source Cartridge Reader（OSCR）はSA-1を読む必須要件として、PIC12F629にsnesCICを書き込んで実装することと、Si5351によるクロック供給を挙げている。nesdevのフォーラムにも「21.477MHzがSA-1の認証にとって決定的だった」とある。

本装置ではそのどちらも使っていない。

7.2 実際の構造 — Super FXと同型だった

正しい理解はこうである。**SA-1はクロックを与えなければ眠っており、ROMは素通しで読める**。クロックを与えることで起動してバスを掌握し、自分の状態を返すようになる。つまり**クロックを与えるほど読めなくなる**。

SA-1は「直列だから読めない」ではなかった。眠らせれば素通しになる



この2つは両立しない。カート1番へのクロック配線1本で切り替わる。

図7 クロックの有無で見え方が反転する。眠っているSA-1はROMを素通しにするが、セーブ領域には手が届かない。

これは本報告の4.2節で述べたスターフォックスの結論と**まったく同じ形**である。当時こう書いた——「クロックが無いから通せない」ではなく「**クロックが無いから大人しかった**」が正解だった、と。同じ結論が、別のチップでもう一度現れた。

過去の観測が、すべて逆向きに読み直せる。マスタークロックを供給していた時期、SA-1はバンク\$C0に対して0x02や0x34を返していた。これを「施錠されている」と解釈していたが、実際は**起動したSA-1がバスを握って自分の状態を返していた**。読めなかったのではなく、こちらが起こしていた。

7.3 誤った結論と、その原因

本報告は当初、この章に「**CIC認証も不要だった**」と書いた。それは**誤りである**。原因は実験の順序による交絡であり、記録として残す価値があるので経緯を書く。

誤った根拠は次のものだった。

- ① CIC認証を行うファームで、カービィ成功・再現成功・マリオRPG成功
- ② 認証しないファームに書き換えて「CICなしでも5/6で成功」

②を独立した検証として扱ったが、**独立していなかった**。①でCIC認証を何度も通したあとの状態で②を実行している。SA-1のBW-RAMは電池でバックアップされており、状態が電源サイクルを越えて残る経路が実在する。

表13 交絡を除いた検証 (2026-08-29)

条件	試行	成功
CIC認証なし (一度も認証していない状態から)	16回	0
CIC認証あり	2回	1 (2回目で成功)

順序を変えれば結論が変わる実験を、独立した検証として扱っていた。4章で「1回の観測を結論にするな」と繰り返し述べておきながら、ここでは**観測の独立性**という別の前提を踏み外している。試行回数を増やしても、交絡は消えない。

この誤りは指摘によって発見された。装置の製作者から「昨夜の実験はCIC認証→CICなしの順だったから、SA-1に認証が残っていたのかもしれない」という指摘があり、検証したところ覆った。**実験を設計した側は、自分が置いた順序を疑いにくい**。

7.4 実際に必要だったもの

表14 SA-1の読み出しに必要な条件（交絡を除いた検証にもとづく）

項目	要否	理由
CIC認証	不明	各20回の全ダンプで 40% 対 45%。 差を検出できなかった （7.6節）
起動読み（本番前に短く読む）	必要	無いと 0/20。\$E0 を1バンク読むだけで 18/20 になる
マスタークロック（カート1番）	不要	1～8MHzなら読めるが得はない。21.477MHzは速すぎて有害。 ただしLOWに落としてはいけない（0/29） 。浮かせること
システムクロック（カート57番）	出さない	sanniの手順でも明示的に禁止されている
1回の接続で全64バンクを読み切る	必要	SA-1は時間とともに提示内容を変える。4MBを95秒で読み切る
接触の良いカートリッジ個体	必要	同一タイトルでも個体で結果が変わる
リトライ	必要	最良条件でも全ダンプ成功率は70%前後。3～4回に1回は失敗する

マスタークロックが不要である点は、交絡の影響を受けていない。2026-08-29の検証では、I-RAM窓が単一値であること（＝SA-1が起動していないこと）を確認したうえで読み出しに成功している。

速度が要件になるのは、SA-1が時間とともに提示内容を変えるためである。バンクごとに接続し直す方式では1回の吸い出しに5～7分かかり、その間に状態が変わる。1回の接続で読み切る連続バンクモードにより95秒へ短縮したことで、この問題は消えた。5.3節で述べたOSCRとの128倍の速度差は、ここでだけ実際に効いていた。

7.5 対照実験でわかったこと（355試行）

上の要件は当初、失敗の観察から推定したものだった。2026年9月1日、条件を1つずつ変えて各20回ずつ取り直した。判定は「起動読みの後に\$C0を1バンク読み、既知の正解とバイト単位で一致するか」。

起動読みの中身	成功	95%信頼区間
起動読みなし	0/20（0%）	0-16%
\$C0 を1回（sanni相当）	13/20（65%）	43-82%
\$C0 を6回	13/20（65%）	43-82%
\$E0 を1回	18/20（90%）	70-97%
6バンク全部	18/20（90%）	70-97%

効いているのは回数ではなく、どのバンクを叩くかである。\$C0 を6回叩いても1回と変わらず、\$E0 を1回叩けば6バンク全部と同じ結果になる。

訂正。 本稿では当初「sanniのダミー読みでは足りない」と書いていた。45回連続の失敗が根拠だったが、**実験では65%通る**。真の失敗率が35%なら45連敗の確率は10のマイナス20乗であり、事実上ありえない。**あの45回は、いまとは別の何かは違っていた。それが何かは分かっていない。**

クロックについても同じ方法で測った。ハイインピーダンス 17/20、2.667MHz 16/20、4.000MHz 10/20、**LOW固定 0/20**。カート1番をLOWに落とすと読めなくなり、消費電流も20mA増える。「駆動しない」と「LOWにすること」は違う。従来は配線していなかったため結果的に浮いており、区別が付いていなかった。

窓の寿命も測った。起こしてから0秒・10秒・30秒・60秒・180秒待って読んでも、成功率は80%・80%・80%・80%・73%で落ちない。**窓は閉じない**。「1回の接続で読み切れ」という要件は、窓の寿命が理由ではないことになる。

判定方法そのものも検証した。1バンク判定と全64バンク読みを20回突き合わせ、一致は18/20。外れた2件はどちらも「判定は成功・全体は失敗」という楽観側で、64バンクすべてがROMらしく見えるのにCRCだけ違う型だった。**本節の成功率はすべて約10%の上振れを含む**。基準80%に対し、実際の全ダンプ成功率は70%前後と見るのが妥当

である。

7.6 CIC認証は、結局のところ分かっていない

本稿は当初「CIC認証は不要」と書き、次に「必要」と訂正した。その訂正の根拠は「CICあり4回中4回成功、CICなし6回中1回成功」だった。2026年9月2日、**全64バンクをバイト単位で比べて各20回取り直したところ、差は消えた**。

条件	1バンク判定	全64バンク	95%信頼区間
CICなし	11/20	8/20 (40%)	22-61%
CICあり	13/20	9/20 (45%)	26-66%

それでも本稿は「CIC認証は不要」とは書かない。この問いでは既に3回結論を変えている。そして今回の結果は**差を否定できなかった**だけであり、「同じである」ことを示したものではない。書けるのは「20回ずつでは差が検出できなかった」までである。

なお、CICなしのときに観測された「64バンクすべてがROMらしく見えるのに中身が3割違う」という壊れ方は、**CICありでも同じ頻度で起きた**。あれはCIC認証の効果ではなく、元々ある失敗モードだった。

実際の成功率も、この測り直しで下方修正された。1バンク判定では80~90%だが、**全64バンクを正しく吸えるのは40~60%である**。判定は40回中7回が偽陽性で、楽観側に偏っていた。**SA-1は2回に1回失敗する。リトライ前提の運用しかない**。

スーパーマリオRPGでも同じ手順を20回試し、10回成功した（50%）。カービィと同じ水準であり、**この成功率はカート個体ではなくSA-1の性質である**。失敗した10回はほぼ全て異なるCRCで、しかも64バンクすべてがROMらしく見え、ヘッダの『SUPER MARIO RPG』まで読めた。**弾いたのは総和検査だけだった**。

7.7 「1回の陰性を結論にしない」

成功率が70~90%であることは、判断の方法そのものに影響する。スーパーマリオRPGは1回目の読み出しが施錠状態だった。そこで打ち切っていたら「マリオRPGは読めない」と結論していた。2回目で成功している。

同じ日、筆者は1回の陰性から誤った結論を3回出した（「データ線が断線している」「クロックが届いていない」「カートを挿すとクロックが死ぬ」）。いずれも試行回数を増やしただけで消えた。

確率的な現象に対して、**単発の観測は証拠にならない**。これは4章で扱った「決定的か確率的かを見分ける」の裏返しである。決定的な誤りは1回の観測で捕まえられるが、確率的な現象は**陰性を何回積んでも陰性の証明にならない**。

7.8 セーブデータは、逆に取り出せない

ROMとは対称の問題が残った。**セーブデータ（BW-RAM）はSA-1が調停しており、SA-1が動作していなければ読めない**。

クロックを与えない → SA-1は眠る → ROMが読める／BW-RAMには届かない
 クロックを与える → SA-1が起きる → BW-RAMが読める／ROMはSA-1に握られる

両立しない。カート1番へのクロック配線1本の抜き差しで切り替わる。

表15 BW-RAM窓（\$6000-\$7FFF）の応答

条件	I-RAM窓	BW-RAM窓	判定
クロック無・CIC無	0x00一色	0x00一色	SA-1が眠っている
クロック無・CIC有	0x11一色	0x11一色	同上
クロック有・CIC有	228種	204種	SA-1が起動。実データ

SA-1を起動させた状態で読み出した8KBは、**ゲーム自身が初期化する空セーブと同じ位置に同じ構造**を持っており、本物のセーブデータであることは間違いない。しかしゲームに読ませると拒否され、空データで初期化される。

読みごとの相違が8192バイト中27～166バイトあり、多数決でも詰めきれていない。セーブデータはROMと違い、1バイト違えば全体が無効になる。原因が読み取り誤差なのか、**起動したSA-1が自分でBW-RAMを書き換えているのか**は未検証である。同一の電源投入内で連続2回読んで比較すれば判別できる。

なお、スーパーマリオRPGはSRAM 32KBだが窓は8KBであり、残りはSA-1のレジスタ \$2224

でブロックを切り替える必要がある。**/WR**

を+5Vに直結した読み出し専用機では書き込めないため、**原理的に到達できない**。カービィはSRAM 8KBなので窓1枚で全体が入る。

7.7 先行実装との関係

外部ダンパーでSA-1に対応していると明記されているのはOSCRだけであり、そのソースは全て読んだ。本装置はそれとは別の解に到達したことになる。

表16 SA-1に対する2つの解

	OSCR (sanni)	本装置
方針	SA-1を 起動させて 正規の手順で通す	SA-1を 眠らせたまま ROMだけ抜く
必要な追加装備	PIC12F629 (snesCIC) + Si5351	なし
ROM読み出し	可能	可能
セーブデータ	可能 (SA-1が動作しているため)	不可

両者は優劣ではなく**目的が違う**。SA-1を起動させるOSCRの方針は、セーブデータまで扱える点で本装置より広い。本装置の方針は、ROMだけでよいなら追加部品ゼロで済む点で単純である。

「追加で一つも部品を購入しない」という制約から出発した本プロジェクトが、結果として**追加部品を必要としない解**に到達したのは偶然ではないかもしれない。部品を足せる立場にあれば、足す方向の解が先に見つかる。

8. 考察

8.1 制約が構造を決めた

1.2節で述べたとおり、Nano 3台という構成は消去法で選ばれた。I/Oが足りないから分けるしかなかったのであって、設計思想があったわけではない。しかしその結果として**役割ごとに専任のマイコンを持つ構造**が生まれた。

CICは自前のクロックを持たず、与えられたパルスの個数で状態を進める（7.2節）。1ビットあたり372パルスを1つも取りこぼさずに送る必要がある。3.072MHzなら1ビット周期は121us、1ラウンド16ビットで1.94ms、16ラウンド完走で31msである。**この間、数msのブロッキングも許されない**。

表17 CIC担当を確保する構造の比較

	構成	並行する仕事	CIC担当
OSCR (sanni)	Mega 1枚に集約。I/Oは足りる	SDカード書き込み・表示・USB	別途PIC12F629が必要
本装置	Nano 3枚に分散。I/Oのため仕方なく	Nano-3は本来バンク生成のみ	3台目がそのまま担える

MegaはSDカードへの書き込み・表示更新・USB処理を抱えており、us精度のパルス生成を並行できない。だから専用チップを足している。本装置は「1枚では足りない」という弱みのおかげで、CIC専任のマイコンが**追加コストゼロ**で手に入っていた。

強みがそのまま弱みになり、弱みがそのまま強みになった。ただし正確に言えば、これは先見性ではない。Nano-3は本来バンクアドレス生成係であり、CIC役は後から兼務させたものである。制約が生んだ構造がたまたま適合したというのが正確なところである。

8.2 そして同じ罠を、規模を小さくして踏んだ

SA-1実験のためNano-3にCIC役とバンク生成役を兼務させたところ、**割り込み競合でバンクアドレスを2回壊した。**
loop() が PORTB

をリードモディファイライトで回している隙間にPCINT0のISRが入り、ISRが書いたバンク上位2ビット (PB0, PB1 = A22, A23) を古い値で上書きしていた。バンク \$C0 以降はこの2ビットが両方1なので、潰れると \$00-\$3F のまったく別のバンクを読むことになる。

**実測での現れ方: 「前回と98%相違 (64,106/65,536) 」の頻発／bank_196.bin と bank_197.bin
が完全に同一／ヘッダの補数対がNG**

ピンが重複していないことは確認していたが、同じポートレジスタを割り込みとメインループの両方が触る点を見落としていた。重複はピン単位ではなくレジスタアクセス単位で見る必要がある。精密なタイミングを要する仕事は専任させること——OSCRのMegaが陥ったのと同じ罠を、規模を小さくして踏んだ。

8.3 誤診に共通する形

本報告に記録した誤診は、いずれも同じ形をしている。**もっともらしい説明が先に立ち、それが正しいかを測らずに次の実験へ進む。**そして測っていないので、その後の実験がすべて無効になる。

表18 誤診の一覧と、気づいた契機

誤診	気づいた契機
データ化けは電源／デカコン／電圧降下のせい	毎回きっかり30,966バイトという決定性
Super FXはバスを支配するので読めない	安定化電源を直結したら相違0になった
読むたび化けるのは電源不足（ヨッシー）	数分の間に0xFFが12倍動いた（15,306→1,268）
32KB周期の繰り返しはA15の配線不良	はんだ付けをやり直しても変化なし。LoROMの正常な挙動だった
バンク0の前後半が違う＝HiROM	前半は全部0x00で死んでいただけ。8KB刻みのゼロ率で判明
1バイトずれるのはストローブの数の問題	補正したら2バイトずれた。真因はホスト側の受信バッファ未クリア
CICクロック入力は生きている（波形が静止）	測定器側のバグ。生値を記録したら クロックとリセットが短絡 していた
短絡を直したら安定Highになった＝成立	数時間後にチップが焦げた。安定Highは 動けなくなった部品の沈黙 だった
Nano-1のアドレスが進まないのはファームか配線	受け手にパルスを数えさせて切り分け。 ソケットに挟まった金属片 だった
高速化の本丸はNano-1のdigitalWrite	実装は最適化済み。 古いコメントが残っていた だけだった
sanniはSA-1にCIC認証をしていない	握手はコードでなく 別チップ(PIC12F629) が独立してやっていた

特に重い失敗を2つ挙げる。ひとつは**測定系そのものが壊れていた**件である。CIC認証の「成立」判定を5回作り直しては、すべて失敗時に条件を満たされた。原因は判定基準ではなく測り方で、`analogRead()` は1回に約100usかかり、相手が数十us単位で動いているなら意味のあるしきい値は存在しない。ADCをフリーランニングにして約6.5us間隔で900サンプルを持ち帰る簡易ロジックアナライザを作って初めて、**クロック入力とリセット入力が短絡**していたことが分かった。この時点で、それ以前の実験は全て無効になった。

もうひとつは**規定外で動かした部品を信用した**件である。短絡を修正した直後、リセット出力は「Highで安定、351msにわたり遷移ゼロ」になった。これは最初に「成立の証拠」として定義した条件そのものだったが、カートは何も返さず、数時間後にチップは焦げた。抜いて測るとクロック入力・リセット入力・GNDの3本が導通しており、入力段が内部で焼損していた。「直った」と「間に合った」は別の話である。

8.4 本装置の実用性について

率直に述べると、費用対効果は良くない。ありあわせの部品で動かすには適しているが、安価に上げたいのであれば既製品や他プロジェクトを参照した方がよい。ただし4.1節の解決以降は一発で通るようになったため、初期の「1カートあたり20回以上吸わないとまともなデータが得られない」状態からは大きく改善した。

設計上の反省として、**相手がどう振る舞うか分からない信号線をマイコンの端子へ直結したのは甘かった**。片側がLowに固定された線に電流が流れ込む形になり得る。直列抵抗を入れるべきだった。

9. 結論と今後

追加部品を購入しないという制約のもとで、Arduino Nano 3台によるSFCカートリッジ読み出し装置を製作し、15タイトルの読み出しと検証に成功した。特殊チップのうちSuper FXとDSP-1には対応した。

本報告の主たる成果は装置そのものではなく、「読むたび内容が変わる」という**単一の症状を4つの原因へ分離する手続き**である。「相違バイト数の段階に対する単調性」と「0xFF率の安定性」という2指標で、電源不足・接触不良

・タイミング余裕不足・バス調停を区別できることを実測で示した。この手続きは装置に依存せず、他のカートリッジ読み出し装置にも適用できる。

SA-1については2タイトルの読み出しに成功した。**必要だったのは、文献が必須としている装備を用意することではなく、それらを一切与えないことだった。**クロックを与えなければSA-1は眠り、ROMは素通しで読める。先行実装がCIC認証を要求するのは、その実装がクロックを供給する設計だからであり、設計選択が自身の要件を作っていた。

残る未達はSA-1カートリッジの**セーブデータ**である。BW-RAMはSA-1が動作していなければ読めず、ROMの読み出しとは両立しない。実データの取得までは到達したが、byte単位で正確な複製には至っていない。

9.1 次に行うべきこと

優先	項目	根拠
1	SA-1のセーブデータを正確に取る	実データまでは到達している。読みの誤差か、起動したSA-1自身による書き換えかを、同一電源投入内の連続2回読みで判別する（7.6節）
2	読み出しのさらなる高速化	シリアル送信の10us/バイトは物理下限。残る26usの間接費を削る（UDR0直書き、読み出しと送信のパイプライン化）。SA-1では速度がそのまま成功率に効く
3	他のSA-1タイトルでの検証	本報告の結論は2タイトル・3個体で確認したものである。一般性の主張には母数が足りない

なお本報告の結論には、明示しておくべき限界がある。「CIC認証もマスタークロックも不要」は**本装置・2タイトル・3個体で確認した事実**であり、他の環境や他のSA-1タイトルで同じとは限らない。とくにカートリッジ個体差の影響は無視できず、カービィは3枚目で初めて成功している。

付録A 装置の諸元

項目	内容
制御ボード	Arduino Nano (ATmega328P、16MHz、5V) ×3
カートリッジコネクタ	ジャンクのスーパーファミコン本体から摘出した62ピン (実測ピッチ2.50mm、列間隔・キー溝とも非標準)
配線	Nano↔カートはUEW (ポリウレタン銅線)、それ以外はAWG22
基板	2.54mmユニバーサル基板。PCBは使用していない
表示	SH1106 OLED。ハードウェアI2Cピン(A4/A5)はデータバスD4/D5で埋まっているためソフトウェアI2Cで逃がしている
ホスト通信	USBシリアル 1,000,000 bps (Nano-2のみ)
電源	通常はUSBバスパワー1本で動作。消費電流の大きいカートでは安定化電源から5Vバスへ直結が必須
読み出しタイミング	RD=5us / ADDR=5us / PULSE=3us (47us/バイト)。ホストから起動のたびに送るため書き込み直しは不要
CICクロック源	Nano-2 D11(OC2A)。16MHz/(2×(1+1))=4.000MHz、誤差0%
マスタークロック源	摘出水晶+裸ATmega328P (CKOUTヒューズ、lfuse=0xBF) で21.477MHz

付録B OSCRとのクロック設定の比較

OSCRはSi5351を用いる。設定値の単位は0.01Hz (センチヘルツ) である。投入順序も指定されており、CLK2(CIC) → CLK0(マスター) → 最後にCLK1(CPU) となる。

カートピン	信号	OSCRの値	本装置
1	EXT / マスタークロック	21.477272 MHz	21.477 MHz (摘出水晶)
56	CICクロック	3.072 MHz	4.000 MHz (実機旧基板と同値)
57	システム(CPU)クロック	SA-1では出さない	無結線 (一致)
33	REFRESH	Low	Low (一致)
26	RESET	High	High (一致)

CICクロックの周波数について、当初「21.477/7=3.0682MHzは実機と同じ導出」と記録したが誤りだった。CICクロックは映像系 (21.477MHz) ではなく **音声系 (APUの24.576MHz÷8)** から来る。数字が近いのを見て、それらしい説明を後付けしていた。また旧型の実機基板は4MHzでCICを回していた実績があるため、本装置は16MHzから誤差0%で作れる4.000MHzを採用している。

付録C 参考にした資料

- SNESdev Wiki — Cartridge connector.
62ピンカードエッジの完全なピンアサイン。本装置の配線表はこの情報が基になっている。
- NESdev Forums — SNES Edge Connector (t=11389).
「2.54mmではなく2.50mmピッチであり、列間隔とキー溝も非標準」という、自作時に最も重要な指摘を得た。
- sannii — Open Source Cart Reader (GitHub). カートリッジダンパーの事実上の標準実装。ピンアサインとメモリマップ処理の正解を確認する基準として参照した。SA-1関連ではSNES.ino、Discussion #173 (SA-1クロックの校正手順)、公式Wiki「Flashing the snesCIC」を参照している。
- ikari — SuperCIC / snesCIC. supercic-lock.asm および supercic-key.asm。「Key はカートリッジ側、Lock は本体側」という役割の定義、および毎ビットの「両線Low」検査の実装。

- segher — 本家CIC(D411)の逆アセンブル (Neo-Desktop/SNES-CIC-Disassembly)。SuperCICの再実装ではなく本物のチップのコード。命令数を数えて372パルス/ビットを確定した根拠。
- MAME — SM590の実装。CICの命令セットの確定に使用。
- nocash / NESdev Forums (t=19441)。旧型の実機基板がCICに4MHzを供給していたという記述。
- libretro-database / No-Intro。タイトル検索とROMサイズ特定に用いた .rdp データベース (7,621件)。
- Snes9x / RetroArch。ダンプ結果の検証。Snes9xコアの Checksum OK / Invalid Checksum の判定は、「ヘッダだけ正しくて本体が壊れている」状態を発見する決め手になった。
- olikraus — U8g2 (BSD-2-Clause)。SH1106 OLEDの駆動。ハードウェアI2Cピンが埋まっていたためソフトウェアI2C対応が決定的に助かった。
- dumping.guide。SA-1対応の実装に関する調査。

本報告における「確定」と「仮説」の区別について

本文中の数値はすべて実測値であり、装置の記録（README、開発史、CIC実験ノート）および出力ファイルの再検証から取っている。検証していない推論には、その旨を明記した。とくに7.6節のクロック停止仮説と7.7節の制御線仮説は、**因果の筋は通るが未検証である**。

—— 門の前までは来ている。記録は続く。